KRIBU ARCHITECTURE DECISION BRIEF

Why Hermes Agent Cannot Be the Complete KRIBU Platform

A concise fit assessment for the approval-demo and future enterprise product

Decision: Do not adopt Hermes Agent as KRIBU's entire application foundation. Hermes is a capable open-source *personal agent runtime*, while KRIBU requires a governed, multi-user enterprise knowledge platform. Hermes may still be evaluated later as a restricted execution component.

The fundamental mismatch

Hermes is designed around an agent that can use tools, terminals, skills, memory and messaging. KRIBU must securely ingest a company's documents, respect each user's permissions, produce traceable business findings, pause for accountable approvals and maintain an enterprise audit history.

Hermes is strong at

- Model-provider flexibility and agent tool execution
- Skills, plugins, scheduling and subagents
- Streaming runs and human approval pauses
- Sandbox options and memory integrations

KRIBU's core responsibility

- Company data governance and access control
- Evidence-backed analysis with page/cell citations
- Repeatable departmental workflows and approvals
- Multi-company operation, auditing and integrations

Critical gaps

KRIBU requirement	Hermes starting point	Impact
Enterprise tenancy, OIDC, roles and document ACLs	Officially described as a single-tenant personal agent; profiles are not enterprise authorization boundaries.	**Major security redesign**
Authoritative company knowledge	Personal/session memory and optional memory providers, not a versioned document-and-fact system of record.	**New knowledge platform required**
Defensible findings	General document/tool inspection; no complete page/cell provenance and citation lifecycle for KRIBU reports.	**Custom ingestion and evidence layer required**
Reliable business workflows	Flexible autonomous tool loop rather than KRIBU's typed, deterministic milestone/expenditure/issue pipeline.	**Domain workflow still required**
Production operations	Personal-agent storage and broad tool capabilities do not directly provide scalable jobs, tenant isolation or enterprise audit controls.	**Control plane still required**
KRIBU user experience	General agent interfaces rather than source review, findings, timelines, briefings and assigned approvals.	**Product UI still required**

Why a deep fork is risky

- Most difficult KRIBU work remains: data ingestion, permissions, provenance, domain workflows, UI and operations.

- Changing a personal-agent architecture into an enterprise platform creates security and reliability risk.
- Large internal modifications make upstream upgrades and vulnerability fixes harder to adopt.
- Hermes's broad terminal, web and file capabilities increase the attack surface unless aggressively restricted.

Recommended approach

Build KRIBU's control plane with FastAPI, PostgreSQL/pgvector, object storage, OIDC/RBAC, deterministic workflows and a domain-specific Next.js interface. Use a small typed agent layer for the first demo.

KRIBU UI → Secure control plane → Retrieval and workflow → Restricted model/agent runtime

If later needed, test Hermes behind this boundary for selected skills, model routing or sandboxed tools. Hermes must never decide authorization or receive unrestricted access to company systems.

Primary references: Hermes Agent repository, security model, architecture documentation, and MIT license.

Prepared for KRIBU · 14 October 2026